

Informe Técnico de Implementación

Proyecto: CODEFEST AD ASTRA 2026 - Etapa 1

Repositorio: codefest-ad-astra-2026

Ruta del informe: entrega/informe_tecnico.md

Objetivo: documentar, con el mayor nivel de detalle posible, la implementación de la base de conocimiento vectorial, el flujo de extracción y limpieza, el chunking, la indexación con FAISS y el módulo de recuperación que genera `resultados.jsonl`.

1. Resumen Ejecutivo

La implementación del reto se organizó como una tubería reproducible de extremo a extremo:

1. Se inventaria el corpus físico disponible en `data/raw/CODEFEST/`.
2. Se construye un registro canónico de documentos.
3. Se extrae el texto por tipo de archivo con un dispatcher central.
4. Se limpia y normaliza el texto.
5. Se consolida un corpus limpio con `DOC_ID` oficiales de ADL.
6. Se fragmenta el corpus en chunks respetando límites lingüísticos.
7. Se codifican los chunks con `BAAI/bge-m3`.
8. Se construye un índice FAISS `IndexFlatIP` con vectores normalizados.
9. Se guarda la metadata en formato JSON Lines.
10. Se genera `resultados.jsonl` a partir de las 50 consultas de evaluación.

El sistema también incorpora un componente opcional de grafo de conocimiento, almacenado en GraphML, que se fusiona con los resultados vectoriales mediante Reciprocal Rank Fusion (RRF).

2. Estructura General de la Solución

La arquitectura del repositorio se divide en cuatro capas principales:

2.1 Extracción

Ubicada en `src/extraction/`.

Responsabilidades:

- leer archivos heterogéneos del corpus,
- convertir cada formato a texto plano,
- registrar errores sin detener toda la corrida,
- producir `data/processed/<doc_id>.txt`.

Extractores presentes:

- `extract_pdf.py`
- `extract_html.py`
- `extract_json.py`
- `extract_tabular.py`
- `extract_image_ocr.py`
- `extract_pbf.py`
- `extract_mvt.py`
- `extract_txt.py`

2.2 Limpieza y consolidación del corpus

Ubicada en `src/extraction/clean_text.py` y `scripts/build_corpus.py`.

Responsabilidades:

- normalizar Unicode,
- eliminar caracteres de control,
- retirar boilerplate frecuente,
- detectar idioma,
- unir la extracción limpia con el inventario oficial de ADL,
- generar `data/corpus_limpio.jsonl`.

2.3 Chunking e indexación

Ubicada en `src/chunking/` y `src/indexing/`.

Responsabilidades:

- fragmentar cada documento en chunks semánticamente completos,
- calcular tokens,
- codificar los chunks con `sentence-transformers`,
- insertar embeddings en FAISS,
- persistir metadata alineada con el orden interno de FAISS.

2.4 Recuperación y generación de resultados

Ubicada en `src/retrieval/` y `entrega/generador.py`.

Responsabilidades:

- cargar índice y metadata,
- codificar consultas,
- recuperar chunks por similitud coseno,
- integrar el grafo de conocimiento opcional,
- agregar a nivel documento,
- dividir fragmentos largos para respetar el límite de 250 palabras,
- emitir un `resultados.jsonl` con 50 líneas.

3. Dependencias y Requisitos de Entorno

3.1 Python y paquetes

El archivo de dependencias es `requeriments.txt`.

Paquetes principales:

- `pandas`
- `pdfplumber`
- `beautifulsoup4`
- `lxml`
- `openpyxl`
- `pillow`
- `pytesseract`
- `osmium`
- `langdetect`
- `tqdm`
- `mapbox-vector-tile`
- `dotenv`
- `nltk`
- `transformers`
- `sentencepiece`
- `torch`
- `sentence-transformers`
- `faiss-cpu`
- `gdown`
- `networkx`
- `spacy`

3.2 Dependencias de sistema y descargas adicionales

Además de `pip install -r requeriments.txt`, el flujo completo necesita elementos que no están instalados solo con pip:

- Tesseract OCR para poder ejecutar `pytesseract`.
- El modelo de spaCy `xx_ent_wiki_sm` para el grafo y la detección de entidades.
- El recurso de NLTK `punkt_tab` para segmentación oracional.
- Acceso a red la primera vez que `sentence-transformers` descarga BAAI/bge-m3.
- Acceso a Google Drive si se quiere bajar la base vectorial ya construida.

Comandos recomendados en macOS:

```
brew install tesseract
python -m spacy download xx_ent_wiki_sm
python -m nltk.downloader punkt_tab
```

Si Tesseract ya está instalado por otro medio, basta con que `pytesseract` pueda encontrar el binario en el `PATH`.

4. Datos de Entrada y Archivos que Deben Estar Disponibles

4.1 Corpus crudo

El pipeline asume que el corpus original está disponible bajo:

`data/raw/CODEFEST/`

Si la ubicación real es distinta, puede apuntarse con la variable de entorno `DIRECTORIO_CRUDO`.

4.2 Inventario oficial de ADL

Archivo obligatorio:

- `data/raw/CODEFEST/Indice_Datos_Codefest.xlsx`

Este archivo se usa para mapear rutas del corpus al `DOC_ID` oficial de ADL.

4.3 Archivos auxiliares del proyecto

- `data/inventory.csv`
- `data/doc_registry.csv`
- `data/idiomas.csv`
- `data/extraction_errors.csv`
- `data/corpus_limpio.jsonl`
- `data/chunks/chunks.jsonl`

4.4 Base vectorial y resultados

La base vectorial final vive en:

`entrega/base_vectorial/encoder_bge_m3/`

Archivos esperados:

- `index.faiss`
- `metadata.jsonl`

Si se usa el componente bonus del grafo:

- `entrega/base_vectorial/grafos/grafos.graphml`

El archivo de salida de la etapa de recuperación es:

- `entrega/resultados.jsonl`
-

5. Flujo de Construcción Paso a Paso

5.1 Preparar el entorno

Desde la raíz del repositorio:

```
cd /Users/pc/Desktop/Other\ stuff/codefest2026/codefest-ad-astra-2026
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Validaciones recomendadas:

```
python --version
python -c "import faiss, torch, sentence_transformers, spacy, nltk; print('ok')"
```

Si el entorno reporta errores de modelo o recursos faltantes, ejecutar antes del resto:

```
python -m spacy download xx_ent_wiki_sm
python -m nltk.downloader punkt_tab
```

5.2 Inventariar el corpus físico

El inventario recorre `data/raw/CODEFEST/` y produce `data/inventory.csv` con ruta, extensión, tamaño y fenómeno asociado.

Comando:

```
python scripts/inventory.py
```

Qué hace:

- detecta todos los archivos regulares del corpus,
- omite `.DS_Store`,
- normaliza la ruta a formato POSIX,
- extrae el fenómeno desde el nombre del directorio,
- escribe `data/inventory.csv`.

Si el corpus está en otra carpeta:

```
DIRECTORIO_CRUDO=/ruta/al/corpus python scripts/inventory.py
```

Salida esperada:

- `data/inventory.csv`
-

5.3 Construir el registro documental

El registro agrega un `doc_id` interno por archivo inventariado.

Comando:

```
python scripts/build_doc_registry.py
```

Qué hace:

- ordena el inventario por ruta,
- asigna `DOC-0000`, `DOC-0001`, etc.,
- calcula `formato` a partir de la extensión,
- normaliza `fuentes` con barras `/`,
- escribe `data/doc_registry.csv`.

Salida esperada:

- `data/doc_registry.csv`
-

5.4 Validar extracción con una muestra

Antes de ejecutar todo el corpus, conviene correr una muestra aleatoria reproducible.

Comando:

```
PYTHONPATH=$(pwd) python scripts/test_dispatcher_muestra.py
```

Qué hace:

- toma 20 filas aleatorias de `data/doc_registry.csv`,
- guarda `data/doc_registry_muestra.csv`,
- ejecuta el dispatcher sobre esa muestra,
- escribe resultados en `data/processed_muestra/`.

Salida esperada:

- `data/doc_registry_muestra.csv`
- `data/processed_muestra/*.txt`

Si el script reporta problemas de importación, el prefijo `PYTHONPATH=$(pwd)` asegura que la raíz del repositorio quede disponible para resolver `src`.

5.5 Ejecutar la extracción completa

La extracción centralizada vive en `src/extraction/dispatcher.py`.

Comando:

```
python -m src.extraction.dispatcher
```

O, si se prefiere ejecutar el script auxiliar:

```
python scripts/test_dispatcher_muestra.py
```

Qué hace el dispatcher:

- lee `data/doc_registry.csv`,
- decide el extractor por formato,
- normaliza rutas con separadores de la plataforma actual,
- distingue entre PBF de OpenStreetMap y Mapbox Vector Tiles,
- escribe un archivo de texto por documento en `data/processed/`,
- captura errores en `data/extraction_errors.csv`.

Nota importante sobre PBF:

- si la ruta contiene `/tiles/`, se usa `extract_mvt.py`,
- en caso contrario, se usa `extract_pbf.py`.

Salida esperada:

- `data/processed/<doc_id>.txt`
 - `data/extraction_errors.csv` si hubo fallos
-

5.6 Reprocesar solo los documentos fallidos

Si `data/extraction_errors.csv` contiene filas, se puede reintentar solo esos documentos.

Comando:

```
PYTHONPATH=$(pwd) python scripts/reprocesar_errores.py
```

Qué hace:

- lee `data/extraction_errors.csv`,
- filtra el registro original para recuperar solo los `doc_id` fallidos,
- escribe `data/doc_registry_reintentos.csv`,
- vuelve a ejecutar el dispatcher sobre ese subconjunto.

Salida esperada:

- `data/doc_registry_reintentos.csv`
 - nuevos archivos en `data/processed/`
-

5.7 Limpiar el texto y detectar idioma

Comando:

```
python scripts/clean_all.py
```

Qué hace:

- recorre `data/doc_registry.csv`,
- lee cada `data/processed/<doc_id>.txt`,
- normaliza Unicode a NFC,
- elimina caracteres de control,
- elimina boilerplate típico de páginas y reportes,
- detecta idioma con `langdetect`,
- actualiza el texto limpio en el mismo archivo,
- genera `data/idiomas.csv` con el idioma detectado.

Salida esperada:

- `data/idiomas.csv`
 - actualización in-place de `data/processed/*.txt`
-

5.8 Consolidar el corpus limpio

Comando:

```
python scripts/build_corpus.py
```

Qué hace:

- carga el inventario oficial desde `data/raw/CODEFEST/Indice_Datos_Codefest.xlsx`,
- cruza el registro con `data/idiomas.csv` si existe,
- usa el `DOC_ID` oficial de ADL,
- excluye archivos que no tienen correspondencia oficial,
- excluye documentos fuera de los fenómenos 1, 2 y 3,
- produce `data/corpus_limpio.jsonl`.

Salida esperada:

- `data/corpus_limpio.jsonl`

Observación crítica sobre el identificador documental

La implementación final no usa un `doc_id` inventado como identificador semántico del reto. El documento canónico para evaluación es el `DOC_ID` oficial asignado por ADL en `Indice_Datos_Codefest.xlsx`.

Esto es importante porque:

- el emparejamiento de la evaluación de documentos se hace por **fuentes**,
 - el índice final y la metadata ya se alinean con esos identificadores oficiales,
 - se excluyen registros auxiliares del scraping que no forman parte del corpus evaluable.
-

5.9 Fragmentar en chunks

Comando:

```
python scripts/run_chunking.py
```

Qué hace:

- llama a `build_corpus()`,
- llama a `src.chunking.chunker.run()`,
- escribe `data/chunks/chunks.jsonl`.

Criterios de chunking implementados

Para texto en prosa:

- se segmenta por oraciones,
- se permite solapamiento de 1 oración entre chunks consecutivos,
- el tope objetivo es de 450 tokens por chunk,
- nunca se parte una oración en medio, salvo caso anómalo de una oración individual extremadamente larga.

Para formatos tabulares y geográficos:

- `csv`, `xlsx` y `pbf` se manejan como unidades por fila o bloque,
- no se usa solapamiento,
- cada fila o bloque se preserva como una unidad de fragmentación independiente.

Metadata por chunk

Cada chunk generado incluye:

- `doc_id`
- `chunk_id`
- `fuelle`
- `formato`
- `fenomeno`
- `posicion`
- `num_tokens`
- `texto`
- `idioma`

Salida esperada:

- `data/chunks/chunks.jsonl`
-

5.10 Construir la base vectorial FAISS

Si se quiere reconstruir el índice desde cero:

```
python scripts/build_faiss_index.py
```

Qué hace:

- lee `data/chunks/chunks.jsonl` en streaming,
- codifica los textos con BAAI/bge-m3,
- normaliza embeddings a norma unitaria,
- crea un índice `IndexFlatIP`,
- guarda `index.faiss`,
- escribe `metadata.jsonl` en el mismo orden de inserción.

Parámetros internos relevantes:

- `batch_size = 32`
- `superbatch_size = 2000`
- `out_dir = entrega/base_vectorial/encoder_bge_m3`

Por qué se usa `IndexFlatIP`

Se eligió un índice plano por producto interno porque:

- la normalización convierte el producto interno en similitud coseno,
- evita aproximaciones que podrían degradar la recuperación,
- es coherente con el volumen del reto,
- simplifica la persistencia y la reproducción exacta del ranking.

Embedding usado

- modelo: BAAI/bge-m3
- biblioteca: `sentence-transformers`
- dispositivo: CUDA si existe, luego MPS en Apple Silicon, y por último CPU.

5.11 Corregir el índice con los `DOC_ID` oficiales, si se parte de una versión previa

Existe un script de reparación para alinear una base vectorial previa con la versión final usando el `DOC_ID` oficial, sin recodificar los embeddings.

Comando:

```
python scripts/patch_doc_ids.py
```

Qué hace:

- carga el índice y la metadata previos,
- reconstruye el mapeo por (`fuentes`, `posiciones`),
- compara el texto nuevo con el texto viejo para evitar desalineaciones,
- reutiliza los vectores ya calculados,

- escribe una nueva `index.faiss` y `metadata.jsonl`.

Este paso es útil cuando ya existe un índice antiguo y solo hay que corregir el emparejamiento de identificadores documentales.

5.12 Descargar la base vectorial desde Google Drive

Si no se desea reconstruir el índice localmente, existe una ruta de descarga.

Comando:

```
python scripts/fetch_base_vectorial.py
```

Qué hace:

- descarga la carpeta compartida de Google Drive,
- deja `index.faiss` y `metadata.jsonl` en `entrega/base_vectorial/encoder_bge_m3/`,
- verifica que ambos archivos existan al final.

Carpeta de Drive

La base vectorial se maneja fuera de Git porque es demasiado grande para el repositorio.

Ruta de referencia:

<https://drive.google.com/drive/folders/1h1AdEjigBEWfruZlyr7VcYoIN9UTfGSa>

Verificación de integridad

Si la descarga se hizo correctamente, se pueden comprobar los hashes publicados para la versión corregida:

```
shasum -a 256 entrega/base_vectorial/encoder_bge_m3/index.faiss
shasum -a 256 entrega/base_vectorial/encoder_bge_m3/metadata.jsonl
```

Hashes esperados:

- `index.faiss`: 43c226f6b1fc0c38926afdf4299e453c9dd004fe5663e8a1fbc9966e048c3c6
- `metadata.jsonl`: a4e9f211494a210036b824cd1f9c764e5c22e972ee9d1d30bbb8b171b8338627

Tamaño y cardinalidad de la versión corregida

La versión final corregida reportada en el repo tiene:

- 162,569 vectores,
- 162,569 líneas de metadata.

Nota importante

Hubo una versión previa con un conteo distinto de chunks y `doc_id` inventados. Esa versión no debe usarse para entrega final. La versión actual está alineada con los `DOC_ID` oficiales de ADL.

6. Recuperación y Generación de `resultados.jsonl`

El script de salida vive en:

- `entrega/generador.py`

6.1 Entrada y salida

Entradas:

- `entrega/consultas.txt`
- `entrega/base_vectorial/encoder_bge_m3/index.faiss`
- `entrega/base_vectorial/encoder_bge_m3/metadata.jsonl`
- `entrega/base_vectorial/grafos/grafos.graphml`

Salida:

- `entrega/resultados.jsonl`

6.2 Comando de ejecución

```
python entrega/generador.py
```

6.3 Qué hace internamente

1. Lee y parsea las 50 consultas de `consultas.txt`.
2. Carga el encoder BAAI/bge-m3.
3. Carga el índice FAISS y la metadata.
4. Recupera un pool ampliado de chunks con `k = 150`.
5. Toma los primeros 30 resultados vectoriales (`K_RETRIEVE`).
6. Detecta entidades en la consulta con `spacy`.
7. Consulta el grafo de conocimiento y calcula candidatos adicionales.
8. Fusiona los resultados del grafo con los de FAISS mediante RRF.
9. Agrega chunks a nivel documento por max pooling.
10. Genera los 10 fragmentos finales, partiendo chunks de más de 250 palabras en subfragmentos válidos.
11. Escribe un objeto JSON por consulta.

6.4 Parámetros relevantes del generador

- `K_VALIDACION_GRAFO = 150`
- `K_RETRIEVE = 30`

- TOP_DOCS = 3
- TOP_FRAGMENTS = 10
- MAX_PALABRAS_FRAGMENTO = 250
- RRF_KO = 60

6.5 Formato exacto del resultado

Cada línea de `resultados.jsonl` contiene un objeto JSON con la estructura:

- `query_id`
- `documents`: exactamente 3 objetos, cada uno con `rank` y `doc_id`
- `fragments`: exactamente 10 objetos, cada uno con `rank`, `chunk_id`, `doc_id` y `text`

6.6 Regla de 250 palabras

Si un chunk recuperado supera las 250 palabras:

- se divide en subfragmentos,
- los cortes se hacen solo en límites de oración,
- todos los subfragmentos conservan el mismo `chunk_id` original para trazabilidad.

6.7 Grafo de conocimiento bonus

El grafo se usa como un índice adicional:

- se reconocen entidades en la consulta,
- se buscan nodos relacionados en `grafo.graphml`,
- se recolectan chunks asociados a esas entidades,
- se fusionan con FAISS por Reciprocal Rank Fusion.

El grafo es opcional para la competencia, pero está implementado y conectado al generador.

7. Explicación de las Decisiones de Diseño

7.1 Extracción por formato

La extracción se diseñó como un dispatcher por formato porque el corpus es heterogéneo:

- PDF requiere análisis de páginas y OCR de imágenes,
- HTML requiere remoción de etiquetas y contenido no visible,
- JSON puede mezclar texto y estructura de metadata,
- CSV/XLSX se transforma fila por fila,
- imágenes se procesan con OCR,
- PBF tiene dos variantes internas distintas.

PDF

`src/extraction/extract_pdf.py`:

- usa `pdfplumber` para texto nativo,
- recorta aproximadamente el 8% superior e inferior para reducir encabezados y pies,
- aplica OCR con `pytesseract` a imágenes embebidas relevantes,
- tolera imágenes rotas sin abortar la página completa,
- recupera HTML si un archivo etiquetado como PDF no empieza con `%PDF`.

HTML

El extractor HTML elimina scripts, estilos y elementos de navegación/pie, y conserva texto visible.

JSON

`src/extraction/extract_json.py`:

- busca campos de contenido como `title`, `body_text`, `body_paragraphs`, `content`, `texto`, `resumen`, etc.,
- concatena listas de párrafos respetando el orden,
- filtra secciones demasiado cortas que suelen ser menús o boilerplate,
- si no encuentra campos textuales útiles, aplanar el objeto en pares `clave: valor` para no perder contenido estructurado.

Tabular

`src/extraction/extract_tabular.py`:

- convierte cada fila en una representación textual tipo `columna: valor`,
- separa filas con doble salto de línea,
- conserva la granularidad para el chunking posterior.

Imágenes

`src/extraction/extract_image_ocr.py`:

- normaliza todo a PNG en memoria,
- ejecuta OCR multilingüe (`spa+eng+por`),
- devuelve cadena vacía si el OCR no aporta texto útil.

PBF

`src/extraction/dispatcher.py` distingue:

- OSM PBF real,
- Mapbox Vector Tiles en rutas con `/tiles/`.

Esto evita errores de lectura por usar el parser incorrecto para un contenedor binario distinto.

7.2 Limpieza básica y normalización

`src/extraction/clean_text.py` implementa una limpieza conservadora:

- normalización Unicode NFC,
- eliminación de caracteres de control,
- eliminación de patrones de numeración de páginas,
- eliminación de URLs crudas,
- eliminación de fechas tipo **Enero 2024** o **July 2024** cuando aparecen como boilerplate,
- reducción de espacios múltiples,
- reducción de saltos de línea excesivos.

La detección de idioma usa `langdetect` sobre un prefijo del texto y devuelve `es`, `en` o `pt` cuando es posible.

7.3 Identificación documental oficial

Se decidió usar el `DOC_ID` oficial de ADL como identificador canónico del documento para fines de evaluación.

Motivo:

- la evaluación de documentos se hace contra el inventario oficial,
- el documento del reto especifica que el archivo original es la unidad evaluable,
- los identificadores internos del pipeline no deben convertirse en la referencia final.

En la práctica:

- el inventario local genera un `doc_id` interno temporal,
 - el consolidado final sustituye ese identificador por el `DOC_ID` oficial,
 - los archivos auxiliares que ADL no considera documentos se excluyen del corpus final.
-

7.4 Chunking con completitud lingüística

El chunking se diseñó para cumplir una restricción explícita del reto:

- no cortar oraciones en medio,
- no producir fragmentos inconclusos,

- mantener coherencia semántica.

Estrategia concreta:

- prosa: oración + solapamiento de 1 oración,
- tabular y geográfico: unidad discreta por fila/bloque,
- presupuesto de 450 tokens por chunk.

En otras palabras, el corte es semántico y lingüístico antes que puramente mecánico.

7.5 Encoder y espacio vectorial

Se eligió `BAAI/bge-m3` por ser multilingüe y apto para recuperación densa.

La implementación usa `sentence-transformers` y normaliza embeddings antes de indexar.

Ventajas:

- funciona para español, inglés y portugués,
 - produce vectores comparables entre sí,
 - la normalización permite usar producto interno como similitud coseno,
 - se puede ejecutar en CUDA, MPS o CPU sin cambiar el código.
-

7.6 FAISS y persistencia

Se usa `IndexFlatIP` porque:

- es exacto,
- es reproducible,
- no introduce aproximaciones ni parámetros de cuantización,
- es simple de guardar y cargar con `faiss.write_index` y `faiss.read_index`.

La metadata se guarda aparte en JSONL porque FAISS solo almacena vectores e IDs internos, no el texto original ni campos de trazabilidad.

7.7 Recuperación y agregación

La recuperación implementa dos niveles de salida:

- nivel fragmento,
- nivel documento.

La agregación a documento se hace por max pooling: un documento hereda la puntuación de su mejor chunk.

El grafo de conocimiento se integra como una señal complementaria, no como reemplazo del ranking vectorial.

El método de fusión es RRF, porque:

- combina rankings, no escalas de score,
- es robusto cuando FAISS y el grafo producen listas con distribuciones distintas,
- reduce el riesgo de que una fuente dominen artificialmente a la otra.

8. Artefactos de Salida y Estructura Final de Entrega

La estructura esperada de entrega es:

```
entrega/  
  resultados.jsonl  
  generador.py  
  informe_tecnico.pdf  
  base_vectorial/  
    encoder_<nombre>/  
      index.faiss  
      metadata.jsonl  
  grafo/  
    grafo.graphml    (si se entrega el bonus)
```

En esta implementación:

- `generador.py` ya está listo,
- la base vectorial se almacena en `entrega/base_vectorial/encoder_bge_m3/`,
- el grafo opcional está en `entrega/base_vectorial/grafo/grafo.graphml`,
- el informe técnico se entrega como Markdown en este repo y puede exportarse a PDF después.

9. Comandos Recomendados de Reproducción Completa

Secuencia de ejecución sugerida desde cero:

```
cd /Users/pc/Desktop/Other\ stuff/codefest2026/codefest-ad-astra-2026  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
brew install tesseract  
python -m spacy download xx_ent_wiki_sm  
python -m nltk.downloader punkt_tab  
python scripts/inventory.py  
python scripts/build_doc_registry.py
```

```
python scripts/test_dispatcher_muestra.py
python -m src.extraction.dispatcher
python scripts/reprocesarErrores.py
python scripts/clean_all.py
python scripts/build_corpus.py
python scripts/run_chunking.py
python scripts/build_faiss_index.py
python scripts/patch_doc_ids.py
python scripts/fetch_base_vectorial.py
python entrega/generador.py
```

No siempre es necesario ejecutar todo en una sola pasada:

- si la base vectorial ya fue descargada, se puede saltar `build_faiss_index.py` y `patch_doc_ids.py`,
- si solo se está validando extracción, basta con `test_dispatcher_muestra.py`,
- si solo se está generando la entrega final, basta con `generador.py` y la base vectorial correcta presente en disco.

10. Validación y Verificación

10.1 Validación de extracción

```
python scripts/test_dispatcher_muestra.py
```

Verificar:

- que `data/processed_muestra/` se llenó con `.txt` válidos,
- que `data/extraction_errors.csv` no creció de forma inesperada,
- que las rutas del registro se resuelven correctamente en macOS.

10.2 Validación de chunking

```
wc -l data/chunks/chunks.jsonl
```

Verificar:

- que el archivo existe,
- que el número de líneas coincide con el volumen esperado del corpus.

10.3 Validación de la base vectorial

```
python -c "import faiss; idx = faiss.read_index('entrega/base_vectorial/encoder_bge_m3/index)
wc -l entrega/base_vectorial/encoder_bge_m3/metadata.jsonl"
```

Verificar:

- que ambos conteos coinciden,
- que no hay truncamiento en la descarga desde Drive,

- que la metadata y el índice están alineados.

10.4 Validación del generador

```
python entrega/generador.py
wc -l entrega/resultados.jsonl
```

Verificar:

- que `resultados.jsonl` tiene exactamente 50 líneas,
- que cada línea corresponde a una consulta q001 a q050,
- que cada objeto contiene 3 documentos y 10 fragmentos.

10.5 Validación del grafo bonus

```
python scripts/verificar_bono.py
```

Este script muestra la diferencia entre la respuesta solo con FAISS y la respuesta FAISS + grafo para una consulta de ejemplo.

11. Riesgos, Supuestos y Observaciones Importantes

11.1 Archivos gigantes fuera de Git

`index.faiss` y `metadata.jsonl` no se guardan en Git por tamaño.

Por eso se usan:

- Google Drive,
- `scripts/fetch_base_vectorial.py`,
- checksums SHA-256 para validación.

11.2 Dependencias externas no declaradas en pip

Aunque `requirements.txt` cubre la parte Python, siguen siendo necesarias piezas externas:

- Tesseract,
- modelo de spaCy,
- recurso de NLTK,
- descarga del encoder la primera vez.

11.3 Corpus heterogéneo y documentos auxiliares

El corpus incluye archivos no triviales:

- reportes PDF,
- páginas HTML,
- JSONs con estructura heterogénea,

- tablas,
- imágenes,
- PBF con dos variantes de contenedor.

Además, algunos archivos son auxiliares del proceso de scraping y no siempre deben entrar al corpus evaluable final.

11.4 Precisión del mapeo documental

La principal fuente de error del proyecto no está en el encoder ni en FAISS, sino en el emparejamiento correcto entre:

- ruta original,
 - inventario oficial,
 - DOC_ID final,
 - `chunk_id` estable,
 - resultado de evaluación.
-

12. Conclusión

La implementación construye una base de conocimiento vectorial reproducible y alineada con el reto, con énfasis en:

- extracción robusta por tipo de archivo,
- limpieza conservadora,
- chunking coherente con el lenguaje,
- embeddings multilingües,
- FAISS exacto con similitud coseno,
- recuperación con agregación a nivel de documento,
- salida final conforme al formato JSON Lines exigido.

El flujo puede ejecutarse completo desde datos crudos o puede retomarse desde la base vectorial ya corregida y descargada desde Drive. En ambos casos, el diseño mantiene trazabilidad entre documento, chunk, vector y resultado final.